INSTITUTO DE INFORMÁTICA
Universidade Federal de Goiás

PROCESSAMENTO DE DADOS MASSIVOS · 2026/2 (BIA)

Foota Toota

Trabalho 2: Ingestão em Tempo Real e Deploy de Modelo

Brasileirão ao vivo no Pub/Sub e Dataflow, batch e stream juntos no BigQuery,
e um modelo por liga no Vertex AI servido por uma API no Cloud Run.

Raphael Alves de Lima Soares · André Vinicius Moura Koraleski
Victor Gabriel Ribeiro Jacome · Wagner Victor Alves de Menezes

Repositório: github.com/phael-exe/foota-toota

API em produção: https://modelo-api-3gvoh2dyuq-uc.a.run.app

INSTITUTO DE INFORMÁTICA UFG

UFG UNIVERSIDADE FEDERAL DE GOIÁS

VISÃO GERAL

O que entregamos

1 Ingestão em tempo real

Coletor no Cloud Run Job, disparado a cada 5 min pelo Cloud Scheduler, publica no Pub/Sub só as partidas do Brasileirão que mudaram.

`streaming/producer.py`

2 Processamento em streaming

Dataflow (Apache Beam) valida e limpa cada evento. O que for inválido vai para a quarentena com o motivo.

`streaming/pipeline.py`

3 Batch + stream no BigQuery

Tabelas do stream particionadas por dia; views da gold juntam o histórico do Trabalho 1 com o que chega ao vivo.

`sql/10_stream/06_create_view_partidas_unificadas.sql`

4 Deploy do modelo

Um modelo por liga no Vertex AI Model Registry, servido por uma API FastAPI no Cloud Run, por HTTP e pelo push do Pub/Sub.

`modelo/api.py`

EVOLUÇÃO DA ARQUITETURA

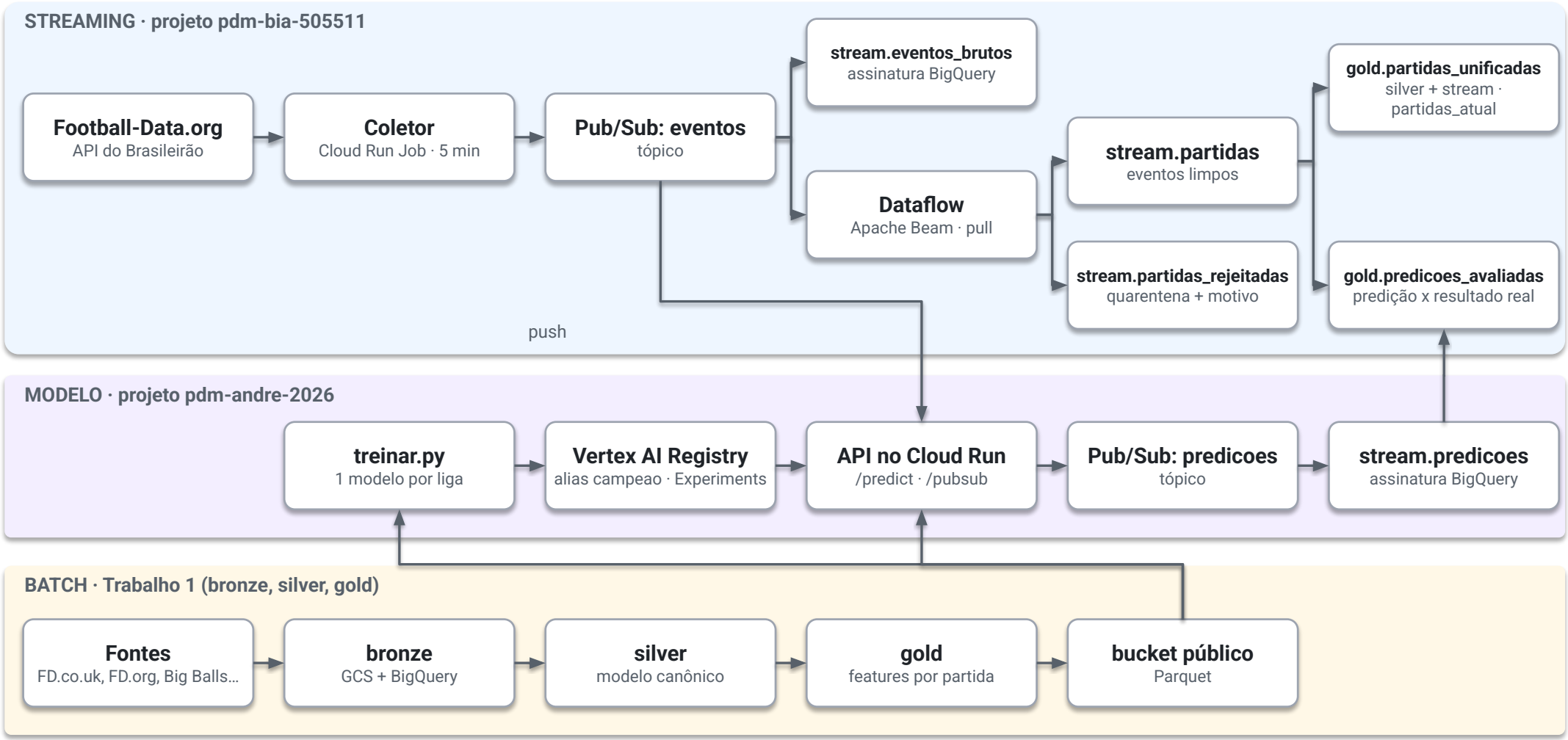
Do Trabalho 1 ao Trabalho 2

	Trabalho 1: batch	Trabalho 2: + tempo real e deploy
Ingestão	scripts baixam as fontes para o GCS (bronze)	+ coletor a cada 5 min publica eventos JSON no Pub/Sub
Processamento	SQL em camadas: bronze → silver → gold, com `ASSERT` bloqueantes	+ Dataflow valida e limpa cada evento; inválidos vão para a quarentena
BigQuery	histórico de todas as fontes em bronze, silver e gold	+ dataset `stream` e views da gold que juntam batch e stream
Modelo	referência em BigQuery ML: regressão logística e boosted trees	um modelo por liga no Vertex AI Model Registry, alias `campeao`
Serviço	previsões gravadas em tabela	API REST em Docker no Cloud Run: `POST /predict` e push do Pub/Sub

- O batch continua igual: a gold dele é a base do treino e o histórico que o stream completa.

COMO OS DADOS ANDAM

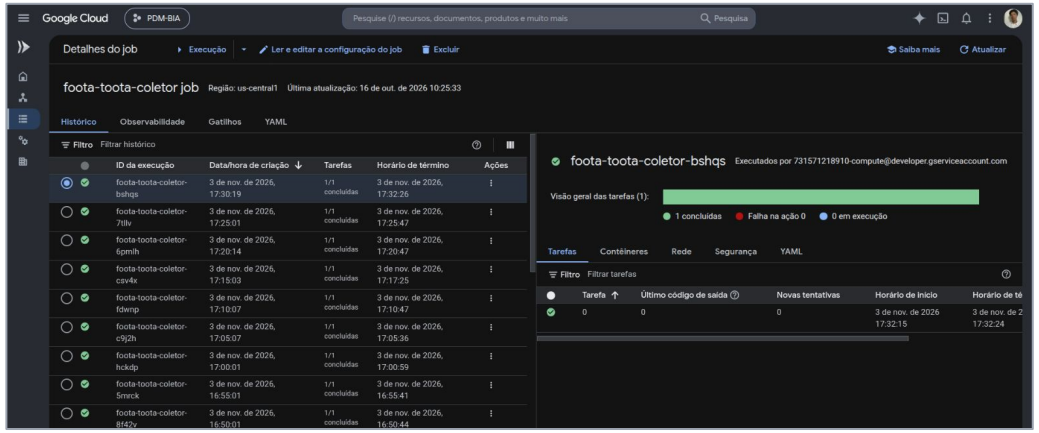
Arquitetura



Clique numa caixa para abrir o código.

Ingestão: o coletor do Football-Data.org

- Roda como Cloud Run Job (`foota-toota-coletor`), disparado pelo Cloud Scheduler a cada 5 minutos (`foota-toota-coletor-5min`).
- Consulta o Brasileirão e publica no tópico `eventos` só as partidas que mudaram: status, placar ou horário.
- O `event_id` identifica aquela versão da partida: `match_id:status:placar:horário`.
- Partida encerrada não volta a agendada: protege do cache antigo da API.
- Modo `replay` reproduz rodadas já jogadas, para demonstrar a qualquer hora.

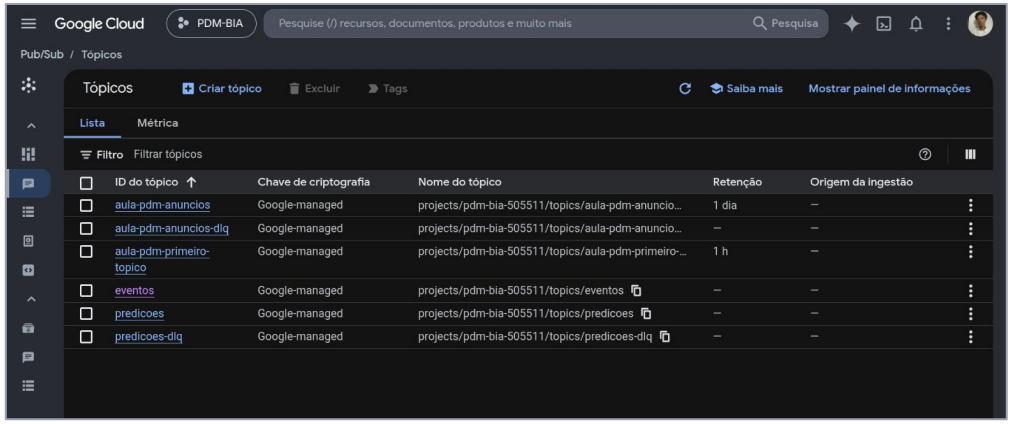


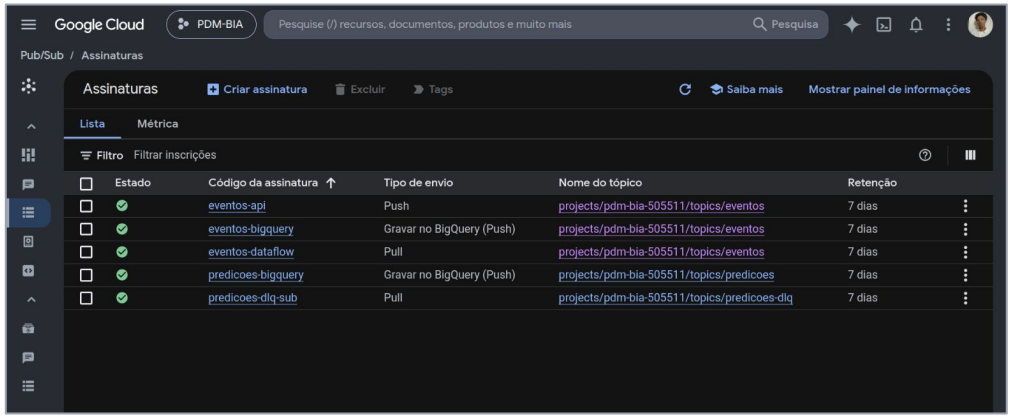


Código: `producer.py` · `deploy_coletor.sh`

Pub/Sub: um tópico, três consumidores

Assinatura	Tipo	Destino
`eventos-bigquery`	BigQuery	`stream.eventos_brutos`: cada mensagem como chegou
`eventos-dataflow`	Pull (guarda 7 dias)	Dataflow → `stream.partidas`
`eventos-api`	Push com token OIDC	API do modelo, `POST /pubsub`
`predicoes-bigquery`	BigQuery	`stream.predicoes` (falhas → `predicoes-dlq`)



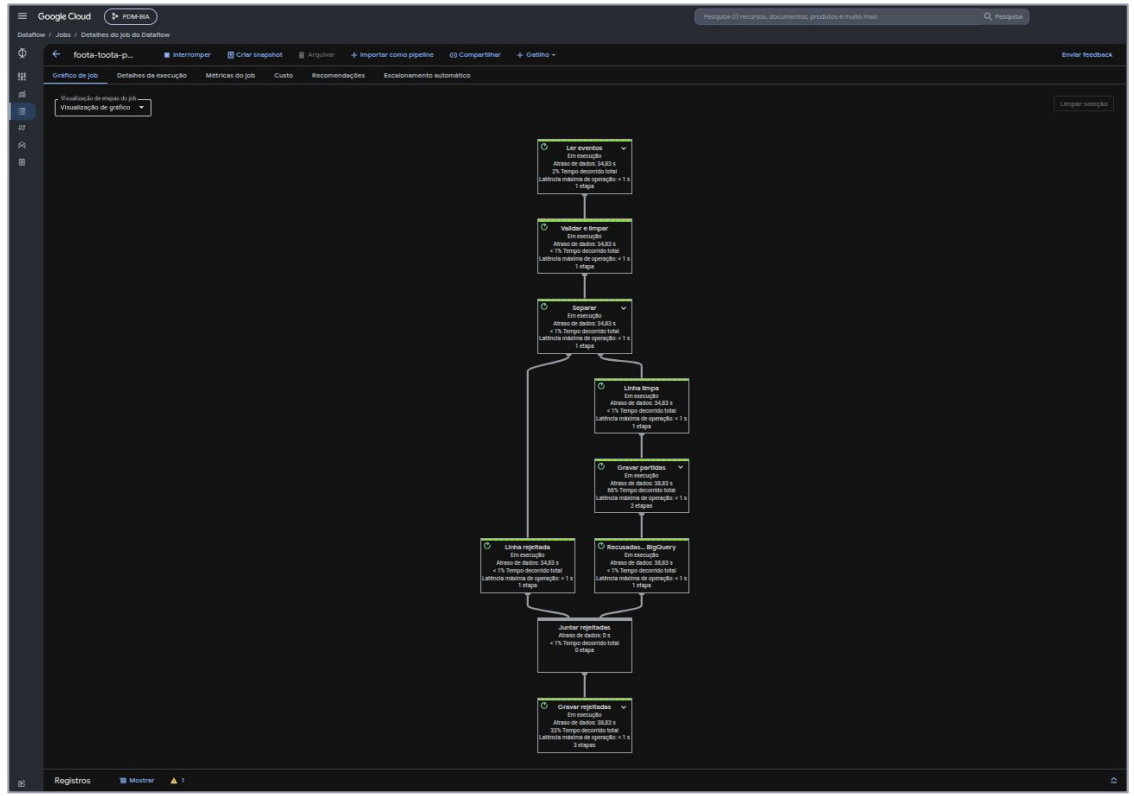


- O bronze do stream não tem código: a assinatura BigQuery grava tudo, mesmo com o Dataflow desligado.
- Ao religar o Dataflow, ele processa o acumulado da assinatura sem perder nada.

Código: `infra.sh`

Dataflow: validação e quarentena

- Recusa o evento
  - sem campo obrigatório
  - competição ou status desconhecido
  - mandante igual ao visitante
  - placar negativo
  - partida encerrada sem placar
- `league_id` e `status` saem no formato da silver.
- O recusado (e o que o próprio BigQuery recusa) vai para `stream.partidas_rejeitadas` com o motivo e a mensagem original.

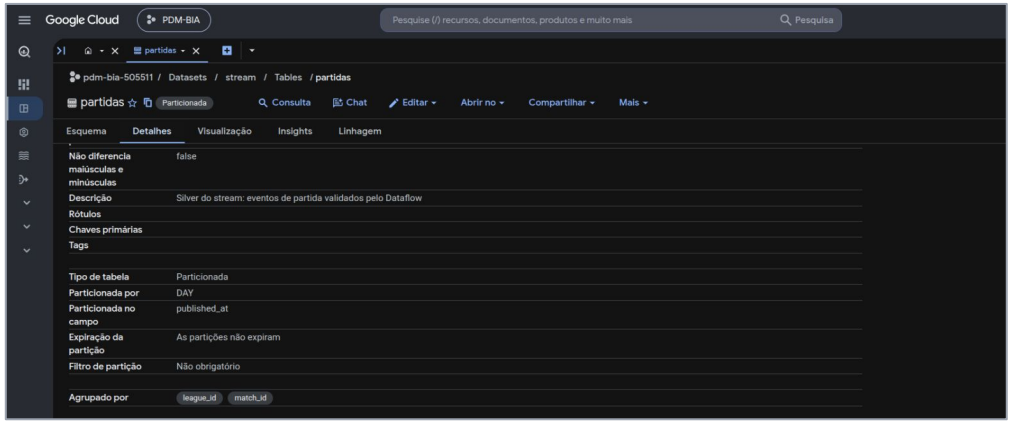


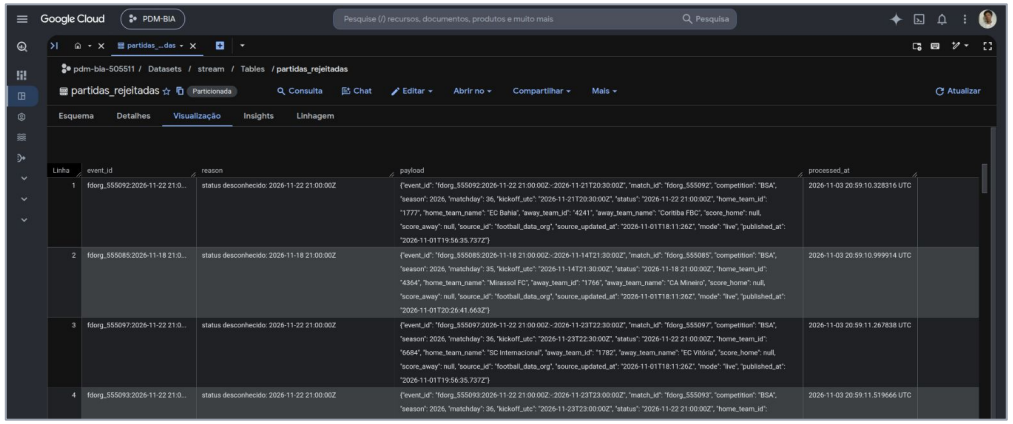

```
$ limpar() sobre o replay da rodada 30 + 5 eventos quebrados
LIMPA      fdorg_555039 scheduled brasileirao_serie_a
LIMPA      fdorg_555039 finished  placar 0-0
LIMPA      fdorg_555038 finished  placar 2-1
REJEITADA mandante e visitante são o mesmo time
REJEITADA placar negativo: -1
REJEITADA partida encerrada sem placar
REJEITADA status desconhecido: ADIADO??
REJEITADA campos ausentes: match_id
```

Código: pipeline.py · dataflow.sh

BigQuery: as tabelas do stream

Tabela	Partição	Cluster	Conteúdo
`eventos_brutos`	dia de publicação	—	cada mensagem como chegou
`partidas`	dia de publicação	liga, partida	eventos limpos
`partidas_rejeitadas`	dia de processamento	—	recusados + motivo
`predicoes`	dia da predição	partida	predições do modelo



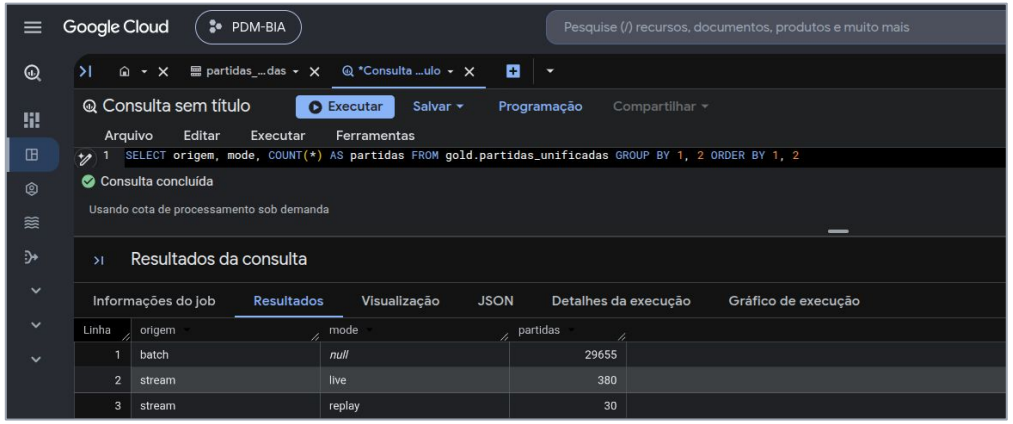


- O stream só escreve na partição do dia; consultas por período leem só as partições do intervalo.
- A página de um jogo lê poucos blocos, por causa da clusterização por liga e partida.

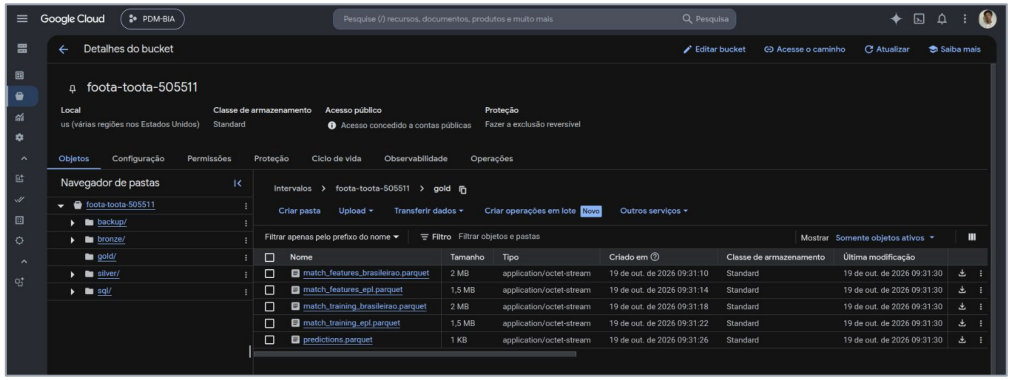
Código: sql/10_stream

Batch e stream juntos

View	O que mostra
`gold.partidas_unificadas`	partidas da silver (`batch`) + último estado de cada partida do stream, com os mesmos ids de clube
`gold.partidas_atual`	o estado mais recente de cada partida: o que a plataforma lê
`gold.predicoes_avaliadas`	cada predição ao lado do resultado real, e se acertou



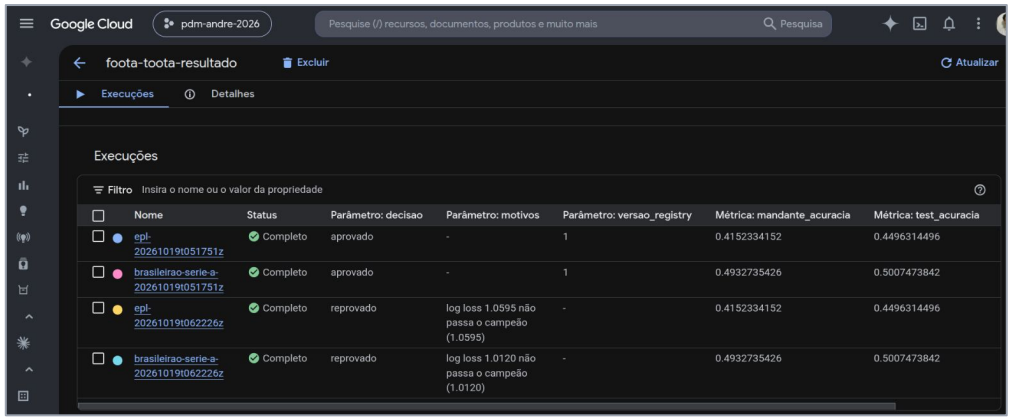
- A versão encerrada vence uma mais nova: a API às vezes devolve uma rodada jogada como agendada.
- Silver e gold também vão em Parquet para um bucket público (31 arquivos, 29.655 partidas): a API e a plataforma leem sem conta no GCP.



Código: [06_partidas_unificadas.sql](#) · [exportar_tabelas_publicas.sh](#)

Modelo: treino e regra de promoção

- Um `HistGradientBoostingClassifier` por liga (Brasileirão e Premier League), sobre a gold pública: casa, empate ou fora.
- Hiperparâmetros pela log loss em EVAL, retreino em TRAIN + EVAL, medida em TEST, que não entra em nenhuma escolha.
- Só é promovido se, em TEST, bater
  - o "sempre mandante" em acurácia
  - as frequências do treino em log loss
  - o campeão atual nas mesmas partidas

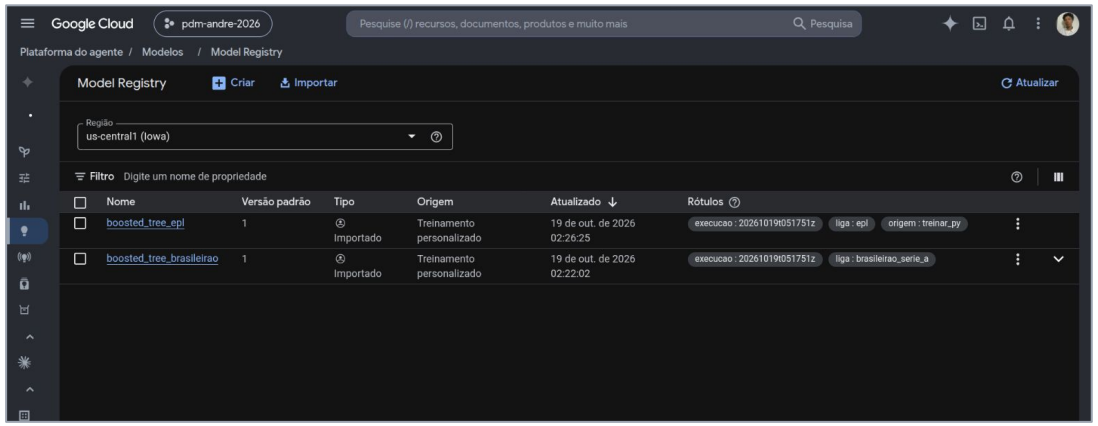


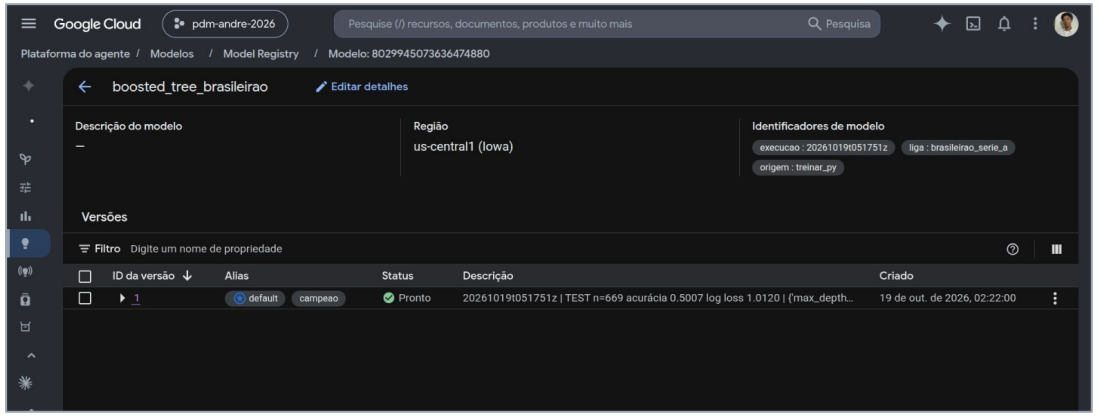
TEST, versão 1	Brasileirão	Premier League
Partidas	669	407
Acurácia: modelo / sempre mandante	50,1% / 49,3%	45,0% / 41,5%
Log loss: modelo / frequências	1,012 / 1,044	1,060 / 1,093

Código: `treinar.py`

Vertex AI Model Registry

- `boosted_tree_brasileirao` e `boosted_tree_epl`, versão 1 com o alias `campeao`.
- A API serve a versão que tem o alias e confere a cada 10 minutos: promover outra versão é mover o alias, sem novo deploy.
- Cada execução do treino, aprovada ou não, vira um run no Vertex AI Experiments.
- Um retreino nos mesmos dados empatou com o campeão e foi reprovado: empate não promove.

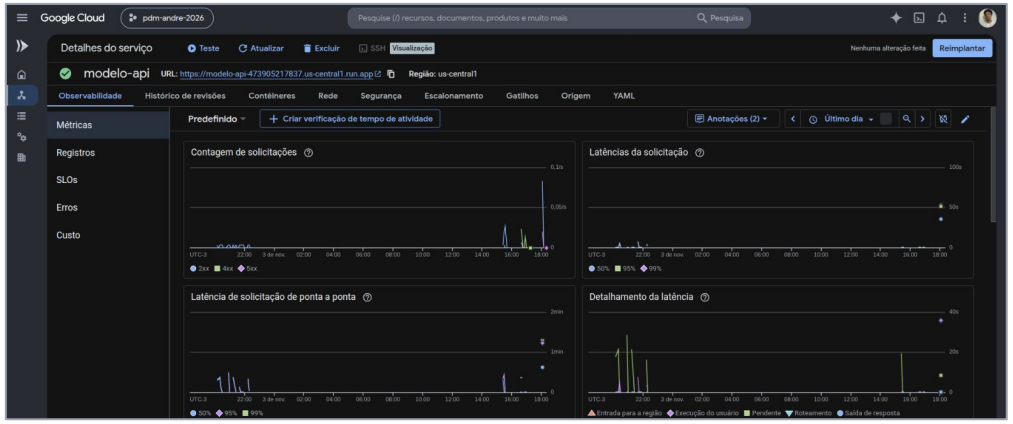




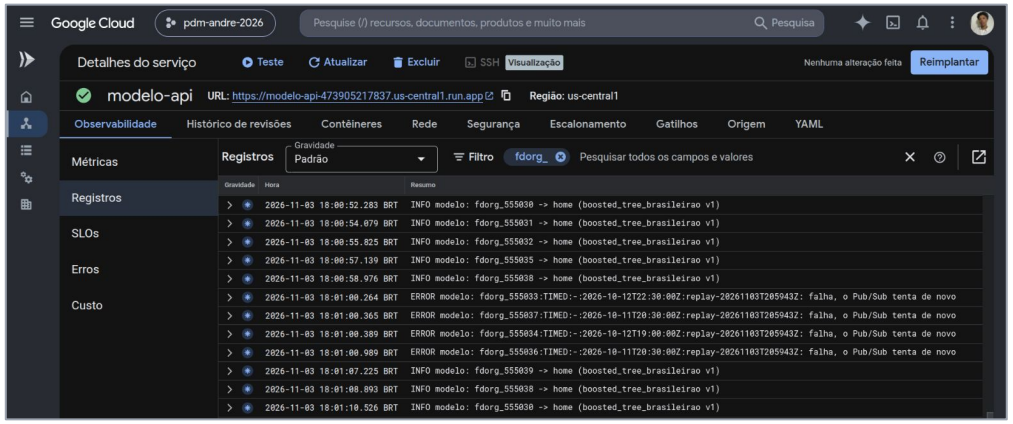
Código: `treinar.py: registrar` · `api.py: Registry`

API do modelo no Cloud Run

Rota	O que faz
`POST /predict`	recebe o `match_id` (ou as features, para simular) e devolve as probabilidades de casa, empate e fora
`POST /pubsub`	recebe o push de `eventos-api`, prevê as partidas agendadas e publica em `predicoes`
`GET /health`	versão do modelo e partidas carregadas de cada liga



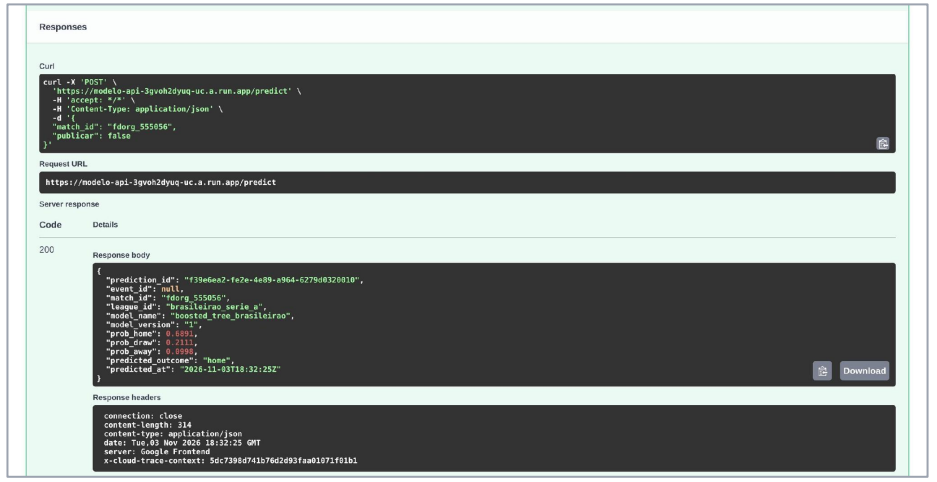
- FastAPI; imagem pelo Cloud Build e Artifact Registry; deploy com `deploy_api.sh`.
- Escala a zero: a 1ª chamada leva ~25 s, as seguintes menos de meio segundo.
- `/pubsub` valida o token OIDC do Pub/Sub: sem token, 401.



Código: `api.py` · `Dockerfile` · `deploy_api.sh` · Swagger

A API respondendo ao vivo (03/11/2026)

```
$ curl -s -X POST https://modelo-api-3gvoh2dyuq-uc.a.run.app/predict \
    -d '{"match_id": "fdorg_555052", "publicar": false}'
{
  "match_id": "fdorg_555052",
  "league_id": "brasileirao_serie_a",
  "model_name": "boosted_tree_brasileirao",
  "model_version": "1",
  "prob_home": 0.6325,
  "prob_draw": 0.1909,
  "prob_away": 0.1766,
  "predicted_outcome": "home"
}
$ curl -s -X POST .../predict -d '{"match_id": "2aa61d11-…"}'   # EPL
  boosted_tree_epl v1: home 0.6502 · draw 0.2074 · away 0.1424
$ curl -s -X POST .../pubsub -d '{}'   # sem token OIDC
  401
```





INTEGRAÇÃO

Do evento à avaliação da predição

1 **Evento agendado**

- o coletor publica em `eventos`; o push entrega em `/pubsub`

2 **Predição**

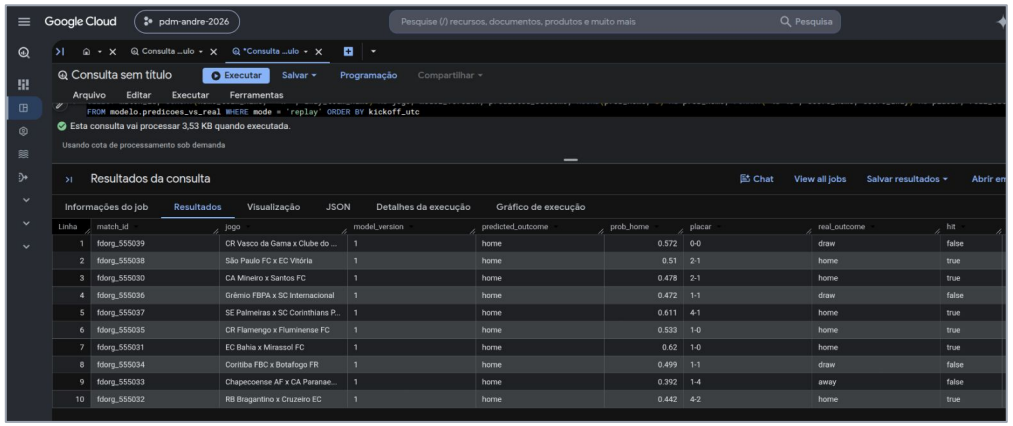
- a API prevê com a versão `campeao` e publica em `predicoes`

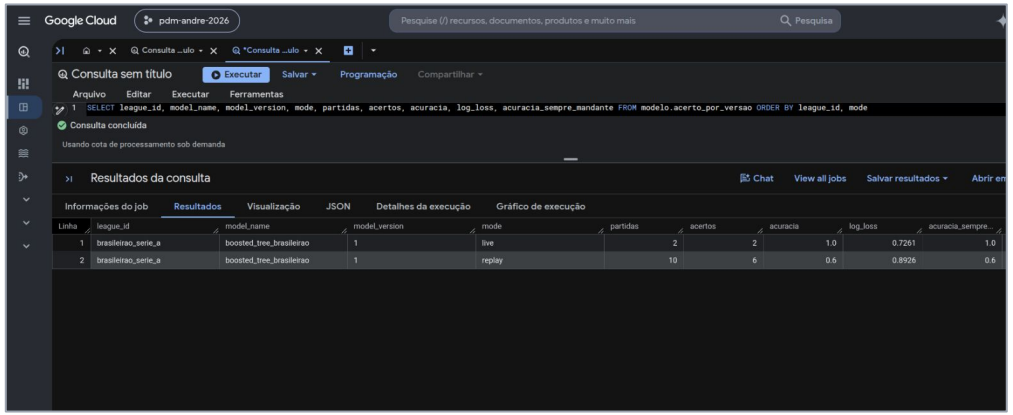
3 **Gravação**

- a assinatura BigQuery grava em `stream.predicoes`

4 **Resultado**

- o placar chega pelo stream e `predicoes_avaliadas` marca o acerto





O Pub/Sub entrega pelo menos uma vez: o prediction_id é o mesmo para o mesmo evento e a mesma versão do modelo, então reentrega não duplica.

Código: `api.py: /pubsub` · `08_predicoes_avaliadas.sql`

POR QUÊ

Escolhas técnicas

Particionamento por dia

o stream só escreve na partição do dia

Cluster pela chave de leitura

liga e partida: a página de um jogo lê poucos blocos

Bronze do stream sem código

assinatura BigQuery grava mesmo com o Dataflow desligado

Quarentena, não descarte

inválidos ficam com motivo e mensagem original

Reentrega sem duplicar

event_id e prediction_id determinísticos

Serverless e barato

job que roda e termina; API escala a zero; Dataflow desligável

ENGENHARIA

Qualidade e reprodutibilidade

- 106 testes automatizados (coletor, pipeline, API e treino), com o HTTP simulado: `uv run pytest`.
- CI no GitHub Actions: lint (`ruff`), tipos (`ty`) e testes a cada push na `main`.
- No batch, regras S1–S23 e G1–G8 como ASSERT bloqueantes, incluindo a prova de zero vazamento temporal.
- Treino reproduzível sem registrar nada: `treinar.py --sem-registrar` aprovou os dois modelos de novo hoje.

```
foota-toota — bash

$ uv run pytest -q
.....................................................................  [ 67%]
.................................                                      [100%]
106 passed in 3.35s
```

```
foota-toota — bash

$ uv run --no-project --with-requirements modelo/requirements-treino.txt python modelo/treinar.py --sem-registrar
INFO httpx: HTTP Request: GET https://storage.googleapis.com/foota-toota-505511/gold/match_features_brasileirao.parquet "HTTP/1.1 200 OK"

== boosted_tree_brasileirao (brasileirao_serie_a)
partidas: TRAIN 8240, EVAL 380, TEST 678
escolhido em EVAL (log loss 1.0311): max_depth=2, learning_rate=0.03, max_iter=174; retreinado em TRAIN+EVAL
TEST                  acurácia  log loss   brier
  modelo                0.5088    1.0112  0.6059
  sempre mandante       0.4926         -       -
  frequências           0.4926    1.0445  0.6285
decisão: aprovado
INFO httpx: HTTP Request: GET https://storage.googleapis.com/foota-toota-505511/gold/match_features_epl.parquet "HTTP/1.1 200 OK"

== boosted_tree_epl (epl)
partidas: TRAIN 3685, EVAL 380, TEST 449
escolhido em EVAL (log loss 1.0092): max_depth=2, learning_rate=0.1, max_iter=28; retreinado em TRAIN+EVAL
TEST                  acurácia  log loss   brier
  modelo                0.4477    1.0556  0.6380
  sempre mandante       0.4143         -       -
  frequências           0.4143    1.0920  0.6612
decisão: aprovado
```

Código: `tests/` · `ci.yml`

COMANDOS

Como reproduzir

```
# streaming (projeto pdm-bia-505511)
streaming/infra.sh              # tópicos, assinaturas, dataset stream, views
streaming/deploy_coletor.sh     # Cloud Run Job + Scheduler (5 min)
streaming/dataflow.sh iniciar   # pipeline no Dataflow
uv run python streaming/producer.py replay --rodadas 28-30 --intervalo 1

# modelo (projeto pdm-andre-2026)
uv run --no-project --with-requirements modelo/requirements-treino.txt \
    python modelo/treinar.py     # treina, avalia, registra no Vertex AI
modelo/deploy_api.sh            # imagem + deploy no Cloud Run

curl -s https://modelo-api-3gvoh2dyuq-uc.a.run.app/health
```

- Repositório
- Contratos do pipeline
- Roteiro da demo do modelo
- API: Swagger



Muito obrigado!
Dúvidas?

github.com/phael-exe/foota-toota